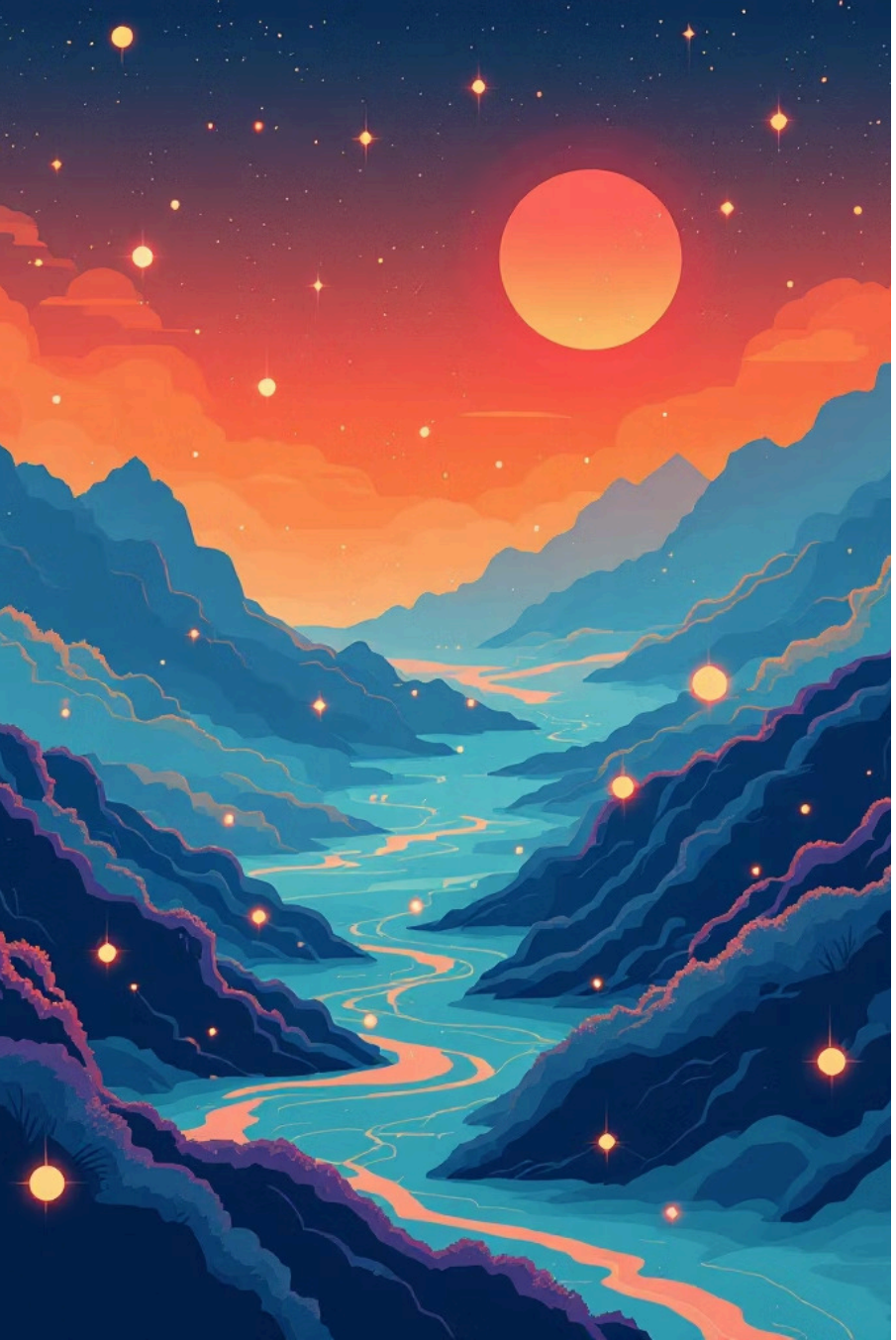


Mastering Arrays & Functions in C

A Practical Guide for Beginner C Programmers

Made with GAMMA

CHAPTER 1

1D Array Problems: Hands-On Practice

Let's dive into practical problem-solving with one-dimensional arrays. We'll explore common tasks and their solutions in C.

Calculate Sum & Average

Understand how to process array elements to derive basic statistics.

Find Extremes

Discover the largest and smallest values within an array efficiently.

Count Even & Odd

Categorize and count numbers based on their divisibility.

Made with GAMMA

Problem: Sum and Average of Array Elements

Logic Explanation

- Initialize a sum variable to zero.
- Iterate through each element of the array.
- Add each element's value to the sum.
- After the loop, divide the sum by the total number of elements to get the average.

C Program

```
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);
    int sum = 0;
    float average;

    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }

    average = (float)sum / n;

    printf("Sum of elements: %d\n", sum);
    printf("Average of elements: %.2f\n", average);

    return 0;
}
```

Sample Output

```
Sum of elements: 150
Average of elements: 30.00
```

Practice Question

Write a C program to find the sum of all positive numbers in a given array.

Made with GAMMA

CHAPTER 2

2D Array Problems: Matrix Operations

Two-dimensional arrays are essentially matrices. Let's explore essential operations and manipulations you can perform.

Matrix Arithmetic

Learn to add, subtract, and multiply matrices.

Transpose & Diagonals

Understand how to reorient a matrix and identify key elements.

Identity Check

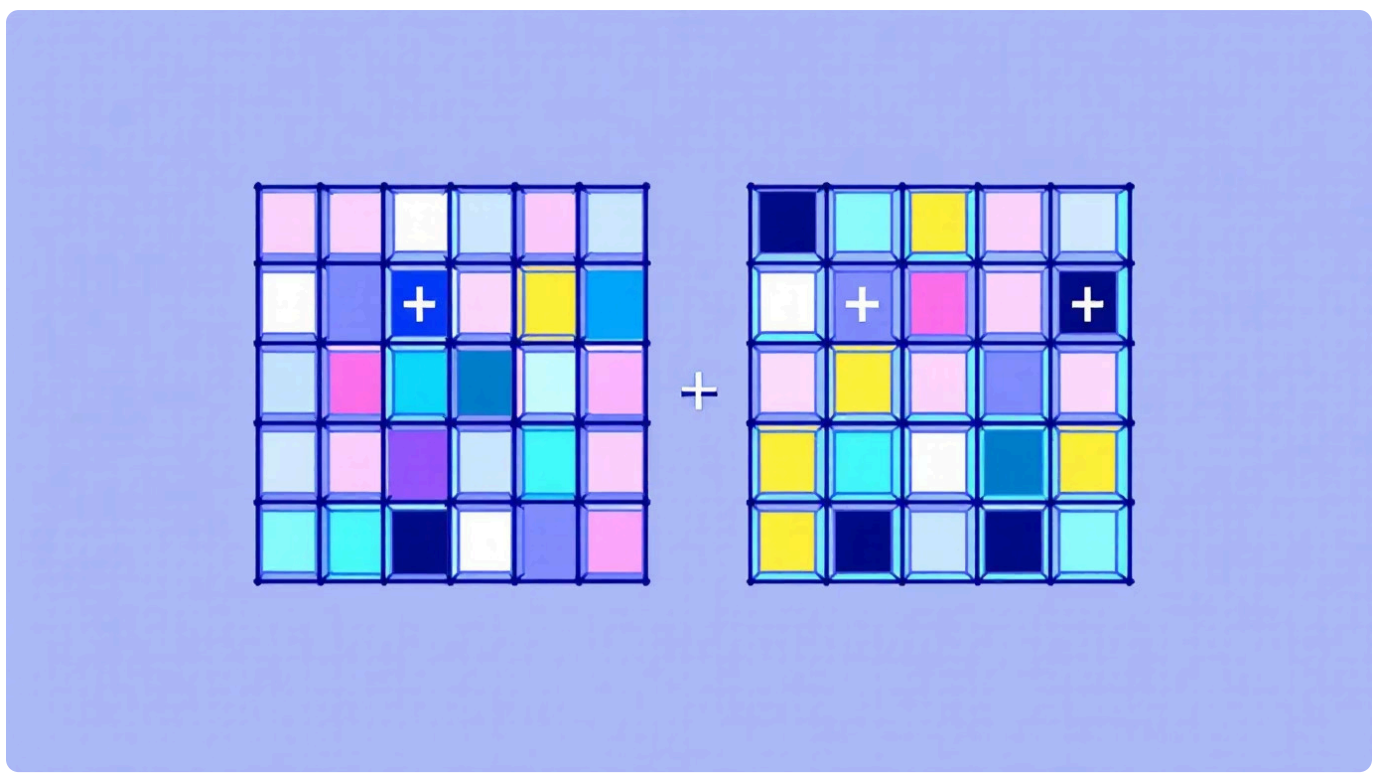
Verify if a matrix meets the criteria for an identity matrix.

Made with GAMMA

Problem: Matrix Addition

Algorithm/Logic Explanation

- Matrices must have the same dimensions for addition.
- Initialize a result matrix of the same dimensions.
- Iterate through each corresponding element of the two input matrices using nested loops.
- Add the elements and store the result in the corresponding position of the result matrix.



C Program

```c
#include <stdio.h>

int main() {
    int A[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int B[2][3] = {{7, 8, 9}, {10, 11, 12}};
    int C[2][3]; // Result matrix
    int rows = 2, cols = 3;

    printf("Matrix A:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", A[i][j]);
        }
        printf("\n");
    }

    printf("Matrix B:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", B[i][j]);
        }
        printf("\n");
    }

    // Adding matrices
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            C[i][j] = A[i][j] + B[i][j];
        }
    }

    printf("Resultant Matrix (A + B):\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", C[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Sample Output

```
Matrix A:
1 2 3
4 5 6
Matrix B:
7 8 9
10 11 12
Resultant Matrix (A + B):
8 10 12
14 16 18
```

Practice Exercise

Modify the program to perform matrix subtraction.

Made with GAMMA

CHAPTER 3

Three-Dimensional Arrays

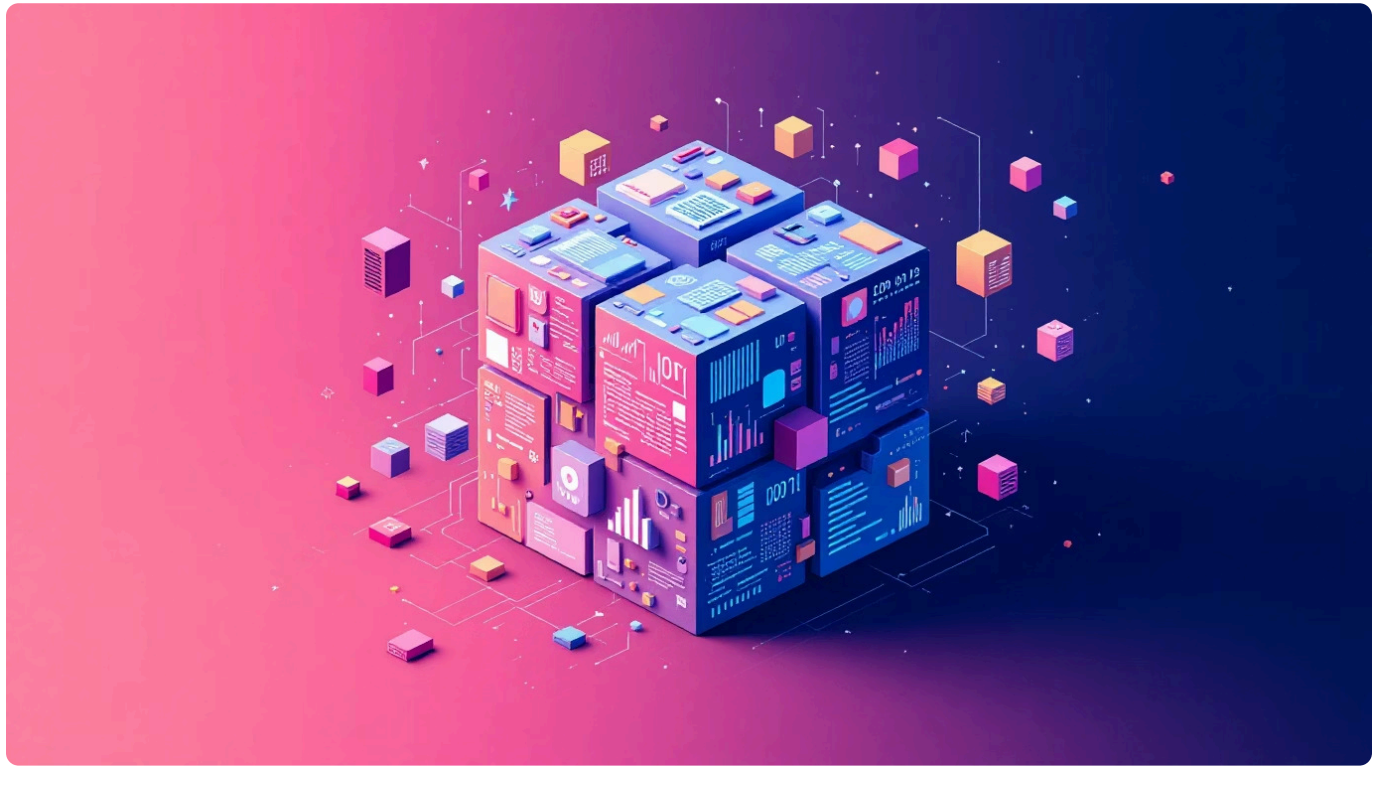
Beyond rows and columns: 3D arrays allow you to model complex, layered data structures, similar to cubes or stacks of matrices.

Brief Explanation of 3D Structure

A 3D array in C is an array of arrays of arrays. Think of it as a collection of 2D arrays (matrices) stacked on top of each other. It's indexed using three subscripts: `array[depth][row][column]`.

Real-life Analogy

Imagine a building with multiple floors (depth), where each floor has rooms arranged in rows and columns. Each room represents an element in the 3D array.



Made with GAMMA

Example Program: Initializing and Traversing a 3D Array

Logic Explanation

- Declare a 3D array with specific dimensions (e.g., 2 "layers", 3 rows, 4 columns).
- Use three nested loops to iterate through each dimension.
- Assign values or print elements using the three indices.

C Program

```
#include <stdio.h>

int main() {
    // Declaring and initializing a 3D array
    int cube[2][3][4] = {
        {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}},
        {{13, 14, 15, 16}, {17, 18, 19, 20}, {21, 22, 23, 24}}
    };

    printf("Elements of the 3D array:\n");
    for (int i = 0; i < 2; i++) { // Iterating through depth (layers)
        printf("Layer %d:\n", i);
        for (int j = 0; j < 3; j++) { // Iterating through rows
            for (int k = 0; k < 4; k++) { // Iterating through columns
                printf("%d ", cube[i][j][k]);
            }
            printf("\n");
        }
        printf("\n");
    }

    return 0;
}
```

Sample Output

```
Elements of the 3D array:
Layer 0:
1 2 3 4
5 6 7 8
9 10 11 12

Layer 1:
13 14 15 16
17 18 19 20
21 22 23 24
```

Practice Question

Write a C program to find the largest element in a 3D array.

Made with GAMMA

CHAPTER 4

Introduction to Functions

Functions are the building blocks of modular C programs. They allow you to organize code, promote reusability, and simplify complex tasks.

Why Functions?

Break down large problems into smaller, manageable pieces.

Modular Code

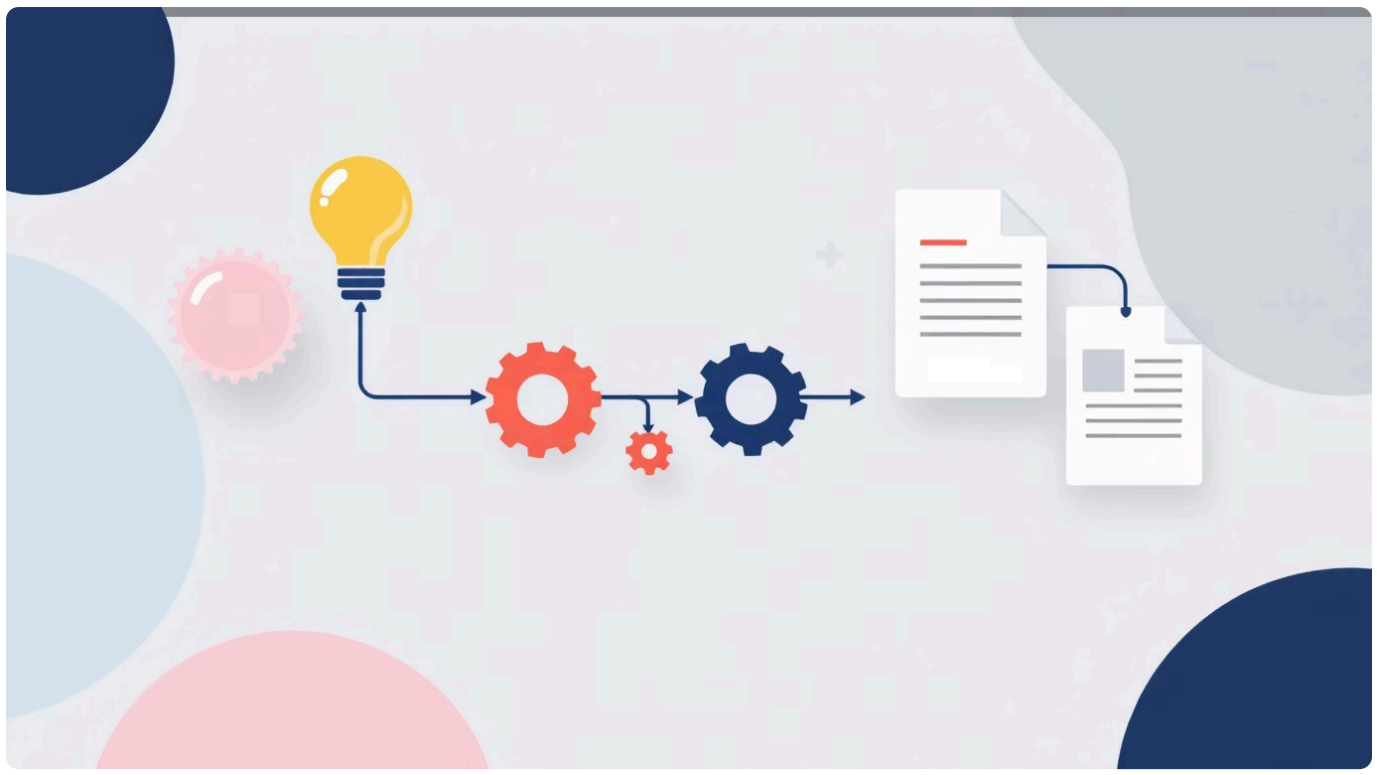
Improve readability and maintainability by structuring your code.

Code Reusability

Write code once and call it multiple times, avoiding repetition.

Made with GAMMA

Function Syntax: Anatomy of a C Function



- **Return Type:** Specifies the type of value the function will return (e.g., `int`, `float`, `void`).
- **Function Name:** A unique identifier for the function.
- **Parameters:** Variables passed into the function (input).
- **Function Body:** The set of statements that perform the function's task.
- **Return Statement:** Optional, sends a value back to the caller.

Example Program: Simple Addition Function

```
#include <stdio.h>

// Function declaration (prototype)
int add(int a, int b);

int main() {
    int num1 = 10, num2 = 5;
    int sum;

    // Function call
    sum = add(num1, num2);

    printf("Sum: %d\n", sum);
    return 0;
}

// Function definition
int add(int a, int b) {
    return a + b; // Returns the sum of a and b
}
```

Sample Output

```
Sum: 15
```

Practice Exercise

Create a function named `multiply` that takes two integer arguments and returns their product. Call this function from `main` and print the result.

Made with GAMMA